

This week

- ACALL, LCALL
- JMP, AJMP, LJMP and SJMP
- Interfacing
 - input/output pins
 - LED
 - Switch
 - 7-Segment Display

JMP, AJMP, LJMP and SJMP

- JMP: Jump (1 Byte, 2 Cycle)
 - 8-bit A + 16-bit DPTR
- SJMP: Short Jump (2 Byte, 2 Cycle)
 - 128 before to 127 after next instruction
- AJMP: Absolute jump (2 Byte, 2 Cycle)
 - 11-bit address (2K Byte block)
- LJMP: Long jump (3 Byte, 2 Cycle)
 - 16-bit address (64K Byte block)

ACALL vs LCALL

- ACALL stands for absolute call while LCALL stands for long call.
- ACALL allows you to jump to a subroutine within the same 2K page while LCALL allows you to jump to a subroutine anywhere in the 64K code space.
- The advantage of ACALL over LCALL is that it is a 2-byte instruction while LCALL is a 3-byte instruction.

Review

- Output:
 - Create a square wave of 50% duty cycle on bit 0 of port 1.
- Input
 - Receive data from P0 and send it to P1

Input Port

- The four 8-bit I/O ports P0, P1, P2 and P3 each uses 8 pins
- All the ports upon RESET are configured as input, ready to be used as input port
- When the first 0 is written to a port, it becomes an output
- To reconfigure it as an input, a 1 must be sent to the port

Example

Port 0 is configured first as an input port by writing 1s to it, and then data is received from that port and sent to P1

```
        MOV     A, #0FFH      ;A=FF hex
        MOV     P0,A          ;make P0 an i/p port
                                ;by writing it all 1s
BACK:    MOV     A, P0         ;get data from P0
        MOV     P1,A          ;send it to port 1
        SJMP    BACK          ;keep doing it
```

Test

- Assume that bit P2.3 is an input and represents the condition of an oven. If it goes high, it means that the oven is hot. Monitor the bit continuously. Whenever it goes high, send a high-to-low pulse to port P1.5 to turn on a buzzer.

Solution

```
HERE:  JNB  P2.3, HERE ;keep monitoring for high
        SETB P1.5      ;set bit P1.5=1
        CLR  P1.5      ;make high-to-low
        SJMP HERE      ;keep repeating
```


Interfacing Switch

- P2.0 as an Input Pin we write logic 1 to it
- PIN P2.1 for LED

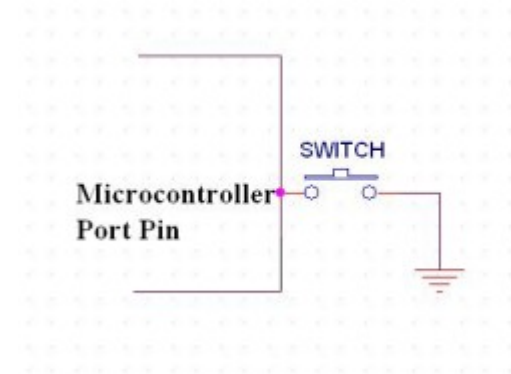
SETB P2.1

SETB P2.0

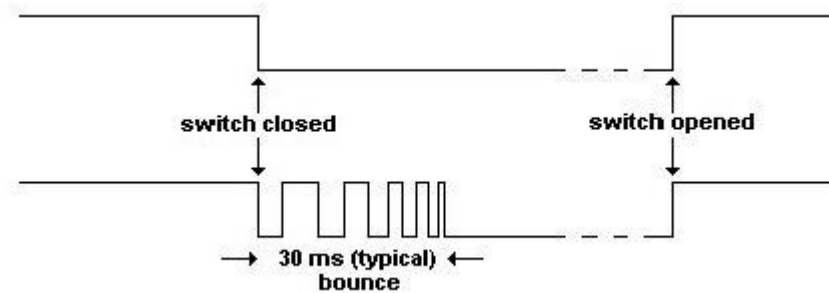
Loop: MOV C,P2.0

MOV P2.1,C

AJMP loop

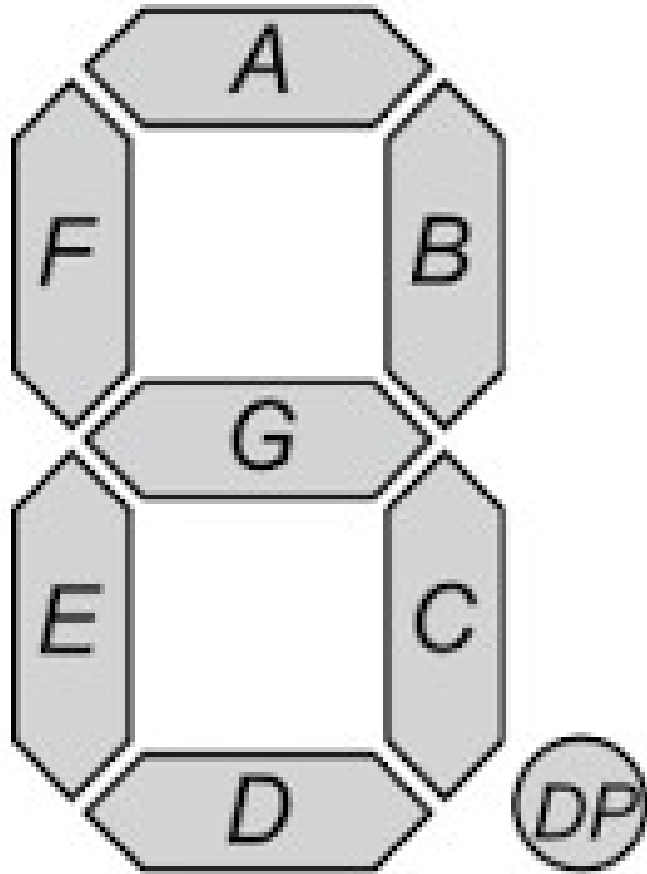


Debouncing



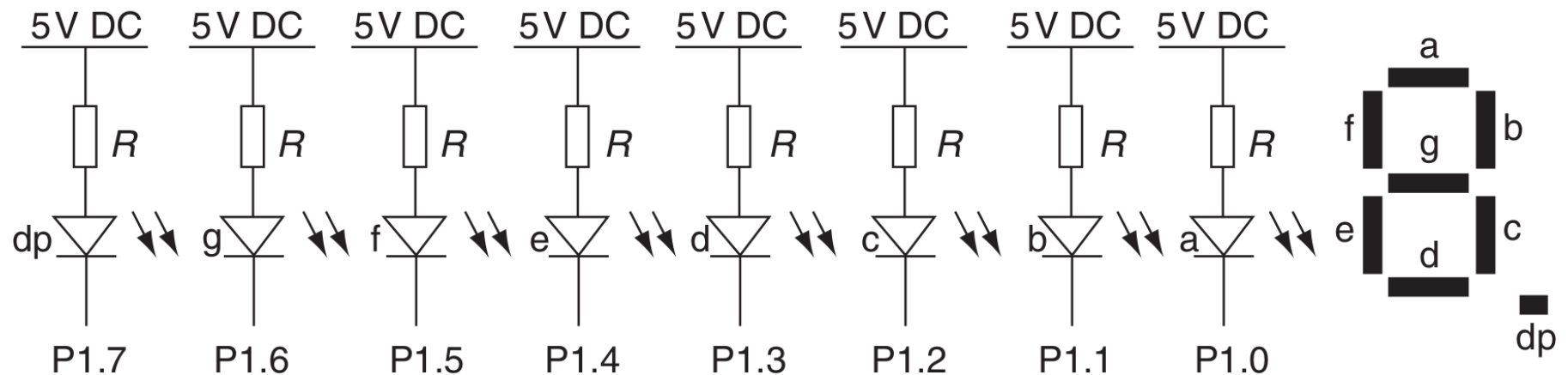
How to solve this problem?

7-Segment Display Interfacing

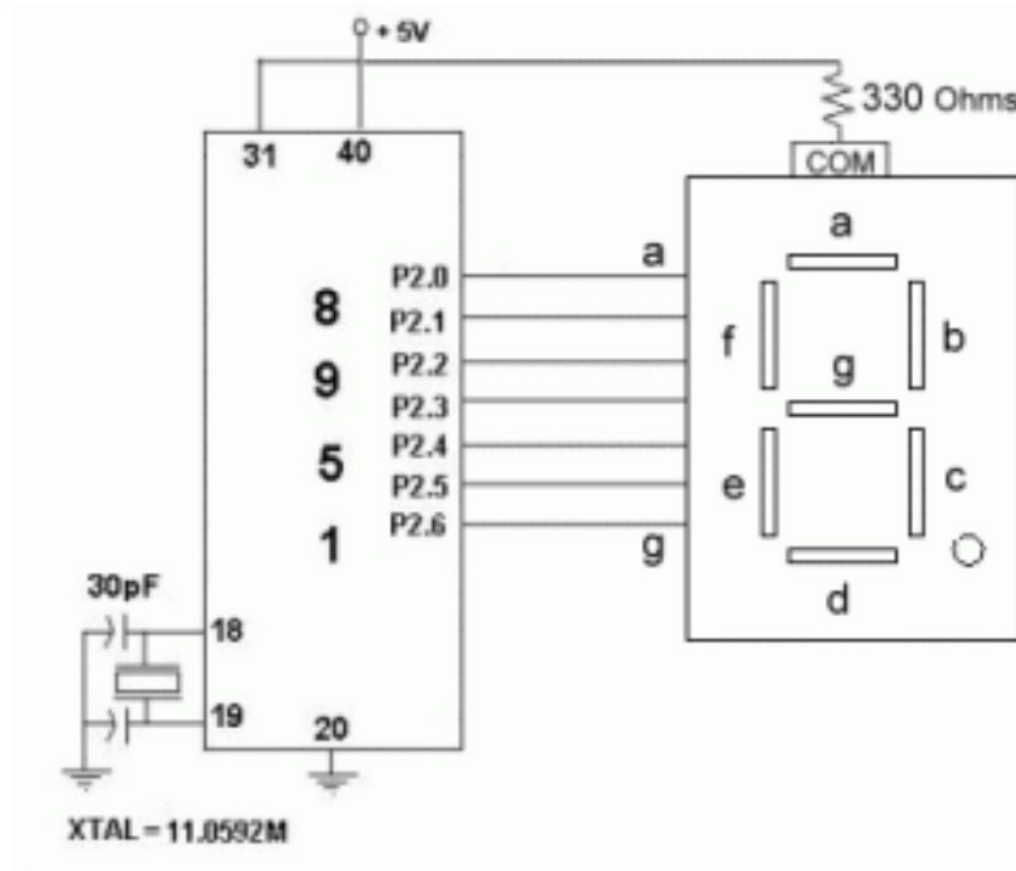


- Common anode
- Common cathode

7-Segment Display Interfacing



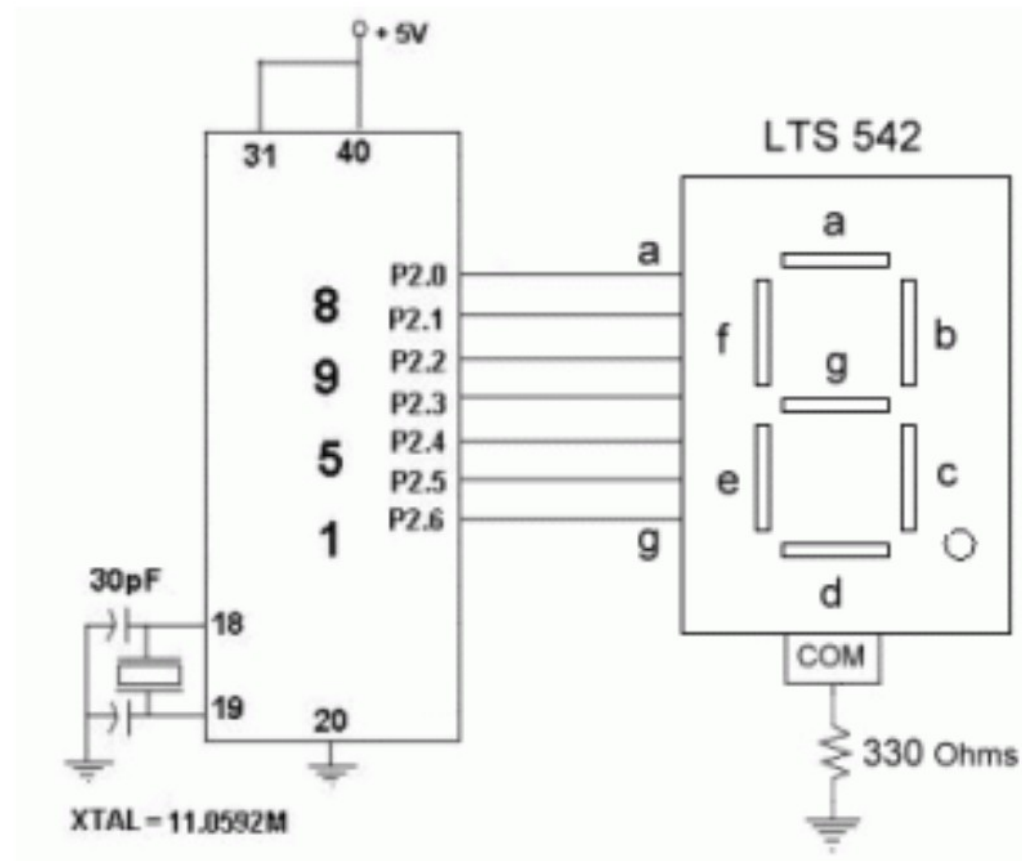
Common Anode



Common Anode

Hex Number	Seven Segment conversion								Seven Segment equivalent
	dot	g	f	e	d	c	b	a	
0	1	1	0	0	0	0	0	0	C0
1	1	1	1	1	1	0	0	1	F9
2	1	0	1	0	0	1	0	0	A4
3	1	0	1	1	0	0	0	0	B0
4	1	0	0	1	1	0	0	1	99
5	1	0	0	1	0	0	1	0	92
6	1	0	0	0	0	0	1	0	82
7	1	1	1	1	1	0	0	0	F8
8	1	0	0	0	0	0	0	0	80
9	1	0	0	1	1	0	0	0	98

Common Cathode



Common Cathode

Hex Number	Seven Segment conversion								Seven Segment equivalent
	dot	g	f	e	d	c	b	a	
0	0	0	1	1	1	1	1	1	3F
1	0	0	0	0	0	1	1	0	06
2	0	1	0	1	1	0	1	1	5B
3	0	1	0	0	1	1	1	1	4F
4	0	1	1	0	0	1	1	0	66
5	0	1	1	0	1	1	0	1	6D
6	0	1	1	1	1	1	0	1	7D
7	0	0	0	0	0	1	1	1	07
8	0	1	1	1	1	1	1	1	7F
9	0	1	1	0	0	1	1	1	67

Simple example

- MOV P1,#0B0H ; display 3
- MOV P1,#99H ; display 4

Test

- Write an 8051 assembly program to display count down (9 – 0) on 7-segment display connected on P1 (Common Anode).

```

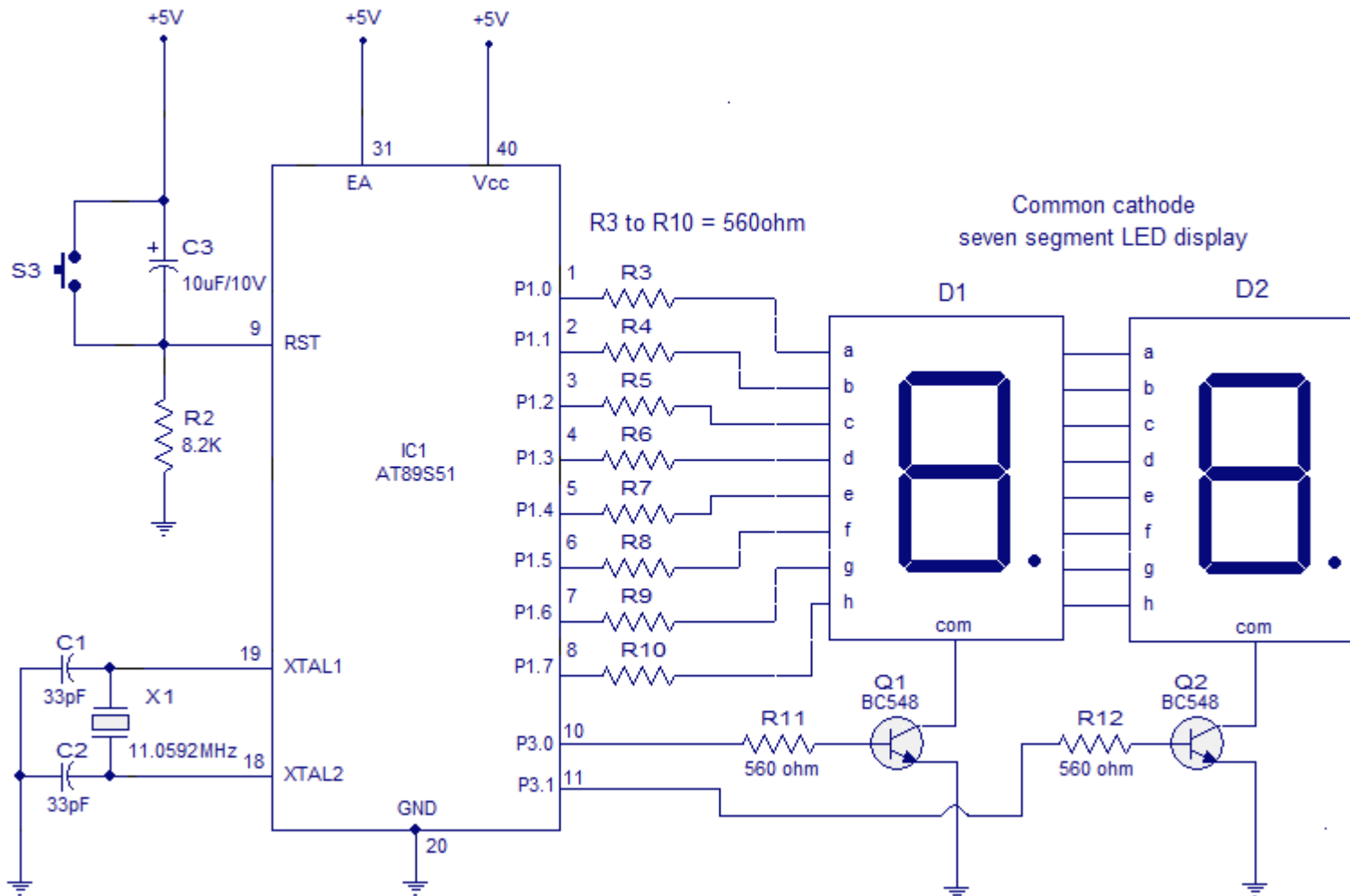
        org 0000h
start:   mov r5,#0ah
        mov dptr,#1000h
up:      clr a
        movc a,@a+dptr
        mov p1,a
        acall delay
        inc dptr
        djnz r5,up
        sjmp start

delay:   mov r0,#0ffh
here1:   mov r1,#0ffh
here:    djnz r1,here
        djnz r0,here1
ret

org 1000h
db 0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x98
end

```

2-digit 7-Segment Display



Multiplexing 7 segment displays to 8051

www.circuitstoday.com

2-digit 7-Segment Display

- Write the asm program to display 16 on 2-digit 7-segment display.